

Sql

Function	Purpose	Example	Output
<code>LENGTH()</code>	Characters count	<code>LENGTH('Tarun')</code>	5
<code>CONCAT()</code>	Strings jodna	<code>CONCAT(name,city)</code>	TarunIndore
<code>SUBSTRING()</code>	Beech ka part	<code>SUBSTRING('Tarun',2,3)</code>	aru
<code>REPLACE()</code>	Text replace	<code>REPLACE('Tarun','a','@')</code>	T@run
<code>TRIM()</code>	Extra spaces remove	<code>TRIM(' Tarun ')</code>	Tarun
<code>LEFT()</code>	Left se characters	<code>LEFT('Tarun',2)</code>	Ta
<code>RIGHT()</code>	Right se characters	<code>RIGHT('Tarun',2)</code>	un
<code>REVERSE()</code>	String ulta	<code>REVERSE('Tarun')</code>	nuraT

</> SQL

-- 1. Name length greater than 5

```
SELECT * FROM student  
WHERE LENGTH(name)>5;
```

-- 2. Longest name

```
SELECT * FROM student  
ORDER BY LENGTH(name) DESC  
LIMIT 1;
```

-- 3. Display "Name - City"

```
SELECT CONCAT(name, ' - ',city)  
FROM student;
```

-- 4. First 3 letters

```
SELECT LEFT(name,3)  
FROM student;
```

-- 5. Last 2 letters

```
SELECT RIGHT(name,2)  
FROM student;
```

1. LTRIM()

Purpose

Left side ke spaces remove karta hai.

 **SQL**

```
SELECT LTRIM('    Tarun');
```

Output

Tarun

Table Example

 **SQL**

```
SELECT  
name,  
LTRIM(name)  
FROM student;
```

2. RTRIM()

Purpose

Right side ke spaces remove karta hai.

SQL

```
SELECT RTRIM('Tarun  ');
```

Output

Tarun

Table Example

SQL

```
SELECT  
name,  
RTRIM(name)  
FROM student;
```

3. MID() / SUBSTRING() ★ ★ ★

MID() MySQL me SUBSTRING() ka alias hai.

SQL

```
SELECT MID('Tarun Patidar',1,5);
```

Output

Tarun

Table Example

SQL

```
SELECT  
name,  
MID(name,1,3) AS short_name  
FROM student;
```

4. LOCATE() ★ ★ ★ ★

Purpose

String ke andar kisi character ya word ki position batata hai.

Syntax

SQL

```
LOCATE(search_string,string)
```

Example

SQL

```
SELECT LOCATE('a','Tarun');
```

5. ROW_NUMBER()

Unique ranking.

```
</> SQL
```

```
SELECT
```

```
name,
```

```
marks,
```

```
ROW_NUMBER()
```

```
OVER(
```

```
ORDER BY marks DESC
```

```
) rank_no
```

```
FROM student;
```

Department Wise

SQL

SELECT

name,

dept_id,

ROW_NUMBER()

OVER(

PARTITION BY dept_id

ORDER BY marks DESC

)

FROM student;

6. OVER()

Window define karta hai.

```
</> SQL
```

```
SELECT
```

```
name,
```

```
marks,
```

```
AVG(marks)
```

```
OVER()
```

```
FROM student;
```

Department Wise

SQL

SELECT

name,

dept_id,

AVG(marks)

OVER(

PARTITION BY dept_id

)

FROM student;

Even IDs

</> SQL

```
SELECT *  
  
FROM student  
  
WHERE MOD(id,2)=0;
```

Odd IDs

</> SQL

```
SELECT *  
  
FROM student  
  
WHERE MOD(id,2)=1;
```

8. LEAD()

Next row ki value.

SQL

SELECT

name,

marks,

LEAD(marks)

OVER(

ORDER BY marks DESC

)

AS next_marks

FROM student;

9. DENSE_RANK() ★ ★ ★

Equal rank

Gap nahi.

SQL

SELECT

name,

marks,

DENSE_RANK()

OVER(

ORDER BY marks DESC

)

FROM student;

10. DISTINCT

Duplicate remove karta hai.

 SQL

```
SELECT DISTINCT city  
  
FROM student;
```

Multiple Columns

 SQL

```
SELECT DISTINCT  
  
city,  
  
dept_id  
  
FROM student;
```

11. WHERE n-1 ★★☆☆

Previous row compare karne ke liye generally `LAG()` use hota hai.

Example

SQL

`SELECT`

`id,`

`marks,`

`LAG(marks)`

`OVER(`

`ORDER BY id`

`) previous_marks`

`FROM student;`



12. CURDATE() - INTERVAL

Current Date se calculation.

10 Days Add

 SQL

SELECT

CURDATE()+INTERVAL 10 DAY;

10 Days Minus

 SQL

SELECT

CURDATE()-INTERVAL 10 DAY;



14. TIMESTAMPDIFF()

Do dates ke beech ka difference nikalta hai.

Syntax

 SQL

TIMESTAMPDIFF(unit,start_date,end_date)



1 Month Add

SQL

SELECT

CURDATE()+INTERVAL 1 MONTH;

1 Month Minus

SQL

SELECT

CURDATE()-INTERVAL 1 MONTH;

1 Year Add

```
</> SQL  
  
SELECT  
  
CURDATE()+INTERVAL 1 YEAR;
```

1 Year Minus

```
</> SQL  
  
SELECT  
  
CURDATE()-INTERVAL 1 YEAR;
```

Table Example

Students added in last 30 days (assuming `admission_date` exists)

```
</> SQL  
  
SELECT *  
  
FROM student  
  
WHERE admission_date>=CURDATE()-INTERVAL 30 DAY;
```

13. CASE WHEN THEN ELSE 🌟

IF ELSE in SQL.

Syntax

```
</> SQL
```

```
CASE
```

```
WHEN condition THEN value
```

```
WHEN condition THEN value
```

```
ELSE value
```

```
END
```

```
</> SQL
```

```
SELECT
```

```
name,
```

```
marks,
```

```
CASE
```

```
WHEN marks >= 90 THEN 'A'
```

```
WHEN marks >= 80 THEN 'B'
```

```
WHEN marks >= 70 THEN 'C'
```

```
ELSE 'Fail'
```

```
END AS Grade
```

```
FROM student;
```

GPT ▾

Years

</> SQL

SELECT

TIMESTAMPDIFF(

YEAR,

'2004-05-10',

CURDATE())

);

Months

</> SQL

SELECT

TIMESTAMPDIFF(

MONTH,

'2024-01-01',

CURDATE())

);

Days

```
</> SQL  
  
SELECT  
  
TIMESTAMPDIFF(  
  
DAY,  
  
'2025-01-01',  
  
CURDATE()  
  
);
```

Student Age Example

Assume `dob` column exists.

```
</> SQL  
  
SELECT  
  
name,  
  
TIMESTAMPDIFF(  
  
YEAR,  
  
dob,  
  
CURDATE()  
  
)  
  
AS age  
  
FROM student;
```

-- 6. First 4 characters using SUBSTRING

```
SELECT SUBSTRING(name,1,4)
```

```
FROM student;
```

-- 7. Replace Indore with Delhi

```
SELECT REPLACE(city,'Indore','Delhi')
```

```
FROM student;
```

-- 8. Reverse every name

```
SELECT REVERSE(name)
```

```
FROM student;
```

-- 9. Remove extra spaces

```
SELECT TRIM(' Tarun ');
```

-- 10. Last 4 digits of phone

```
SELECT RIGHT(phone,4)
```

```
FROM student;
```

Sql

Function	Purpose	Example	Output
<code>ROUND()</code>	Nearest rounding	<code>ROUND(78.56)</code>	79
<code>CEIL()</code>	Hamesha upar	<code>CEIL(78.01)</code>	79
<code>FLOOR()</code>	Hamesha neeche	<code>FLOOR(78.99)</code>	78
<code>ABS()</code>	Absolute value	<code>ABS(-20)</code>	20
<code>MOD()</code>	Remainder	<code>MOD(10,3)</code>	1
<code>POWER()</code>	Power	<code>POWER(2,5)</code>	32
<code>SQRT()</code>	Square root	<code>SQRT(64)</code>	8
<code>RAND()</code>	Random number	<code>RAND()</code>	0.2345...


```
-- 1. Average marks with 2 decimal places
SELECT ROUND(AVG(marks),2)
FROM student;

-- 2. Department-wise rounded average
SELECT dept_id,
ROUND(AVG(marks),2)
FROM student
GROUP BY dept_id;

-- 3. Ceiling example
SELECT CEIL(78.01);

-- 4. Floor example
SELECT FLOOR(78.99);

-- 5. Absolute difference from 80
SELECT name,
ABS(marks-80)
FROM student;
```

-- 6. Even ID students

```
SELECT *  
FROM student  
WHERE MOD(id,2)=0;
```

-- 7. Odd ID students

```
SELECT *  
FROM student  
WHERE MOD(id,2)=1;
```

-- 8. Marks square

```
SELECT name,  
POWER(marks,2)  
FROM student;
```

-- 9. Square root of marks

```
SELECT name,  
SQRT(marks)  
FROM student;
```

```
-- 10. Random 5 students
SELECT *
FROM student
ORDER BY RAND()
LIMIT 5;
```

Difference Table

Function	Purpose	Example	Output	
NOW()	Date + Time	NOW()	2026-07-11 10:45:30	
CURDATE()	Current Date	CURDATE()	2026-07-11	
CURTIME()	Current Time	CURTIME()	10:45:30	
DATEDIFF()	Days Difference	DATEDIFF('2026-07-11', '2026-07-01')	10	
DATE_ADD()	Future Date	DATE_ADD(CURDATE(), INTERVAL 10 DAY)	2026-07-21	
DATE_SUB()	Past Date	DATE_SUB(CURDATE(), INTERVAL 10 DAY)	2026-07-01	
YEAR()	Year	YEAR(admission_date)	2023	
MONTH()	Month	MONTH(admission_date)	7	

```
-- 1
SELECT NOW();

-- 2
SELECT CURDATE();

-- 3
SELECT CURTIME();

-- 4
SELECT DATEDIFF(CURDATE(),admission_date)
FROM student;

-- 5
SELECT DATE_ADD(CURDATE(),INTERVAL 30 DAY);

-- 6
SELECT DATE_SUB(CURDATE(),INTERVAL 30 DAY);
```

```
-- 7
SELECT YEAR(admission_date)
FROM student;

-- 8
SELECT MONTH(admission_date)
FROM student;

-- 9
SELECT DAY(admission_date)
FROM student;

-- 10
SELECT *
FROM student
WHERE YEAR(admission_date)=2023;

-- 11
SELECT *
FROM student
WHERE MONTH(admission_date)=7;
```

SQL

```
-- 12
SELECT *
FROM student
WHERE DAY(admission_date)=15;

-- 13
SELECT name,
DATEDIFF(CURDATE(),admission_date)
FROM student;

-- 14
SELECT name,
DATE_ADD(admission_date,INTERVAL 6 MONTH)
FROM student;

-- 15
SELECT name,
DATE_SUB(admission_date,INTERVAL 1 YEAR)
FROM student;
```

Difference Table

Clause	Purpose	Example
<code>DISTINCT</code>	Duplicate values remove karta hai	<code>SELECT DISTINCT city FROM student;</code>
<code>ORDER BY</code>	Data ko sort karta hai	<code>ORDER BY marks DESC</code>
<code>LIMIT</code>	Rows ko limit karta hai	<code>LIMIT 5</code>

```
-- 1
SELECT DISTINCT city
FROM student;

-- 2
SELECT DISTINCT dept_id
FROM student;

-- 3
SELECT COUNT(DISTINCT city)
FROM student;

-- 4
SELECT *
FROM student
ORDER BY marks DESC;

-- 5
SELECT *
FROM student
ORDER BY marks;
```

```
-- 6
SELECT *
FROM student
ORDER BY name;

-- 7
SELECT *
FROM student
ORDER BY city,marks DESC;

-- 8
SELECT *
FROM student
ORDER BY dept_id,marks DESC;

-- 9
SELECT *
FROM student
LIMIT 5;
```



```
-- 10
SELECT *
FROM student
ORDER BY marks DESC
LIMIT 3;

-- 11
SELECT *
FROM student
ORDER BY marks
LIMIT 3;

-- 12
SELECT *
FROM student
ORDER BY age
LIMIT 1;

-- 13
SELECT DISTINCT city
FROM student
ORDER BY city;
```

```
-- 14
SELECT *
FROM student
LIMIT 5 OFFSET 5;

-- 15
SELECT *
FROM student
ORDER BY marks DESC
LIMIT 1 OFFSET 1;
```

📌 Interview tip

`DISTINCT` aur `GROUP BY` ko log aksar same samajhte hain, lekin ye alag hain:

- `DISTINCT` → Duplicate values hata kar unique values dikhata hai.
- `GROUP BY` → Records ko group karta hai, jisse `COUNT()`, `SUM()`, `AVG()` jaise aggregate functions use kiye ja sakte hain.

Example:

SQL

```
-- Sirf unique cities
SELECT DISTINCT city
FROM student;

-- Har city me kitne students hain
SELECT city, COUNT(*)
FROM student
GROUP BY city;
```



Subquery Type	Returns	Common Operators	Example
Scalar	1 value	=, >, <	<code>marks > (SELECT AVG(marks) FROM student)</code>
Multiple Row	Many rows	IN, ANY, ALL	<code>dept_id IN (SELECT dept_id FROM department)</code>
Nested	Subquery ke andar subquery	Any	Multi-level lookup
Correlated	Har outer row ke liye execute hoti hai	EXISTS, comparison	Department-wise topper
EXISTS	Kam se kam 1 matching row	EXISTS	Department with students
NOT EXISTS	Koi matching row nahi	NOT EXISTS	Department without students

Type	Return Value	Operator
Scalar	1 Row, 1 Column	=, >, <
Multiple Row	Multiple Rows	IN, ANY, ALL
Nested	Multiple Levels	Any
Correlated	Outer Query par depend	EXISTS
EXISTS	True/False	EXISTS
NOT EXISTS	True/False	NOT EXISTS

1. Scalar Subquery

Definition

Sirf 1 row aur 1 column return karti hai.

Example

 SQL

```
SELECT MAX(marks)
FROM student;
```

Output

97

Ye sirf ek value return kar rahi hai.

2. Multiple Row Subquery

Definition

Subquery ek se jyada rows return karti hai.

Iske saath use hote hain

- IN
- ANY
- ALL

Example

 SQL

```
SELECT dept_id  
FROM department  
WHERE block_name='A-Block';
```

3. Nested Subquery

Matlab

Subquery ke andar ek aur subquery.

Level 1



Level 2



Level 3

Example

Pehle department nikalo



Us department ka HOD nikalo



Us department ke students nikalo

</> SQL

```
SELECT *  
FROM student  
WHERE dept_id=  
(  
  SELECT dept_id  
  FROM department  
  WHERE hod_name=  
  (  
    SELECT hod_name  
    FROM department  
    WHERE dept_name='CSE'  
  )  
);
```

4. Correlated Subquery ★ ★ ★ ★ ★

Sabse important.

Definition

Outer query ki har row ke liye subquery dubara execute hoti hai.

Outer Row 1



Subquery



Outer query ki har row ke liye subquery dubara execute hoti hai.

Outer Row 1



Subquery



Outer Row 2



Subquery



Outer Row 3



Subquery



Department Wise Topper

SQL

```
SELECT *  
FROM student s  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

5. EXISTS ★ ★ ★ ★ ★

Definition

Agar subquery kam se kam ek row return kare → TRUE

Nahi kare → FALSE

Departments jinke students hain

SQL

```
SELECT *  
FROM department d  
WHERE EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
);
```



6. NOT EXISTS

Reverse of EXISTS

Departments without students

 SQL

```
SELECT *  
FROM department d  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
);
```

Departments without Female Students

</> SQL

```
SELECT *  
FROM department d  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
  AND gender='Female'  
);
```

Departments without Topper (>95)

</> SQL

```
SELECT *  
FROM department d  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
  AND marks>95  
);
```

1 Highest Marks in Every Department

SQL

```
SELECT *  
FROM student s  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

2 Lowest Marks in Every Department

SQL

```
SELECT *  
FROM student s  
WHERE marks=  
(  
SELECT MIN(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

4 Students Older Than Department Average

SQL

```
SELECT *  
FROM student s  
WHERE age >  
(  
SELECT AVG(age)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

1/C/6a512cab-dcd0-03ee-a37b-da0b2300611a

5 Department Having Maximum Students

SQL

```
SELECT dept_id, COUNT(*) total_students  
FROM student  
GROUP BY dept_id  
HAVING COUNT(*) =  
(  
SELECT MAX(cnt)  
FROM  
(  
SELECT COUNT(*) cnt  
FROM student  
GROUP BY dept_id  
) x  
);
```

6 Students in A-Block Departments

SQL

```
SELECT *  
FROM student  
WHERE dept_id IN  
(  
  SELECT dept_id  
  FROM department  
  WHERE block_name='A-Block'  
);
```


7 Students Not in A-Block

SQL

```
SELECT *  
FROM student  
WHERE dept_id NOT IN  
(  
  SELECT dept_id  
  FROM department  
  WHERE block_name='A-Block'  
);
```

8 Highest Marks Overall

SQL

```
SELECT *  
FROM student  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
);
```

9 Second Highest Marks

</> SQL

```
SELECT *  
FROM student  
WHERE marks=  
(  
  SELECT MAX(marks)  
  FROM student  
  WHERE marks<  
  (  
    SELECT MAX(marks)  
    FROM student  
  )  
)  
);
```

10 Third Highest Marks

SQL

```
SELECT *  
FROM student  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE marks<  
(  
SELECT MAX(marks)  
FROM student  
WHERE marks<  
(  
SELECT MAX(marks)  
FROM student  
)  
)  
);
```

3 Students Above Department Average

SQL

```
SELECT *  
FROM student s  
WHERE marks >  
(  
  SELECT AVG(marks)  
  FROM student  
  WHERE dept_id=s.dept_id  
);
```